

Infrastructure as Code (IaC) & Configuration Management

Infrastructure as Code (IaC): Managing infrastructure through version-controlled configuration files instead of manual, click-by-click setup.

Terraform: A declarative IaC tool that provisions and manages infrastructure resources through provider plugins.

Terraform State: The record Terraform uses to map configuration resources to actual managed objects; it is normally stored locally as terraform.tfstate unless a remote backend is configured.

Variables and Outputs: Variables provide configurable inputs; outputs display or expose useful values after Terraform runs.

Modules: Reusable containers of Terraform resources, variables, and outputs.

Ansible: An agentless automation tool used mainly for configuration management and repeatable operational tasks.

Inventory: A list of managed hosts and their groups; this lab uses localhost.

Playbooks: YAML files containing ordered automation tasks applied to hosts.

Area	Terraform	Ansible
Main purpose	Provision infrastructure resources	Configure systems and run operational tasks
Style	Declarative desired infrastructure	Task-based automation playbooks
State	Uses a state file to track managed resources	Usually does not maintain a Terraform-style infrastructure state file
Typical use	VM, network, cloud resource, DNS creation	Package installation, users, services, configuration files
In this lab	Creates a local file	Installs Nginx, creates a user, starts Nginx

A practical DevOps workflow often uses Terraform first to create infrastructure, then Ansible to configure the operating system and applications on it.

GitHub Repo Link: <https://github.com/nisha-subramaniyan/iac-demo.git>

Installation verification

```
mouli@localhost:~/iac-demo$ terraform version
Terraform v1.16.2
on linux_amd64
mouli@localhost:~/iac-demo$ ansible --version
ansible [core 2.21.4]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/home/mouli/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/mouli/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Jun 19 2026, 12:46:00) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  pyyaml version = 6.0.1 (with libyaml v0.2.5)
```

Main.tf

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

resource "local_file" "lab_file" {
  filename = "${path.module}/${var.file_name}"
  content  = var.file_content
}
```

variables.tf

```
variable "file_name" {
  description = "Name of the file Terraform will create"
  type        = string
  default     = "terraform-created.txt"
}

variable "file_content" {
  description = "Content written into the managed file"
  type        = string
  default     = "Created and managed by Terraform."
}
```

terraform.tfvars

```
file_name      = "nisha-terraform-lab.txt"
file_content   = "This file was created by Terraform using variables."
```

outputs.tf

```
output "created_file_path" {
  description = "Absolute path of the created file"
  value       = local_file.lab_file.filename
}

output "created_file_content" {
  description = "Text written to the file"
  value       = local_file.lab_file.content
}
```

terraform init

```
mouli@localhost:~/iac-demo/terraform$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.5"...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

terraform plan

```
mouli@localhost:~/iac-demo/terraform$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# local_file.lab_file will be created
+ resource "local_file" "lab_file" {
+   content          = "This file was created by Terraform using variables."
+   content_base64sha256 = (known after apply)
+   content_base64sha512 = (known after apply)
+   content_md5       = (known after apply)
+   content_sha1       = (known after apply)
+   content_sha256     = (known after apply)
+   content_sha512     = (known after apply)
+   directory_permission = "0777"
+   file_permission    = "0777"
+   filename           = "./nisha-terraform-lab.txt"
+   id                 = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ created_file_content = "This file was created by Terraform using variables."
+ created_file_path    = "./nisha-terraform-lab.txt"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run
"terraform apply" now.
```

Terraform validate

```
mouli@localhost:~/iac-demo/terraform$ terraform fmt
mouli@localhost:~/iac-demo/terraform$ terraform validate
Success! The configuration is valid.
```

terraform apply

```
mouli@localhost:~/iac-demo/terraform$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# local_file.lab_file will be created
+ resource "local_file" "lab_file" {
  + content          = "This file was created by Terraform using variables."
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename          = ".nisha-terraform-lab.txt"
  + id                = (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ created_file_content = "This file was created by Terraform using variables."
+ created_file_path    = ".nisha-terraform-lab.txt"
```

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.lab_file: Creating...
local_file.lab_file: Creation complete after 0s [id=69a952ec47068e0cdd74140bfda354f8c2c08e49]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

```
created_file_content = "This file was created by Terraform using variables."
created_file_path    = ".nisha-terraform-lab.txt"
```

Terraform Execution

```
mouli@localhost:~/iac-demo/terraform$ cat nisha-terraform-lab.txt
This file was created by Terraform using variables.mouli@localhost:~/iac-demo/terraform$ terraform output
created_file_content = "This file was created by Terraform using variables."
created_file_path    = ".nisha-terraform-lab.txt"
mouli@localhost:~/iac-demo/terraform$ terraform state list
local_file.lab_file
mouli@localhost:~/iac-demo/terraform$ terraform show
# local_file.lab_file:
resource "local_file" "lab_file" {
  content          = "This file was created by Terraform using variables."
  content_base64sha256 = "NSV091YH1upz8FttY4L58E9+FFqGqHgv5WLSPOXLNHM="
  content_base64sha512 = "WtviUGLsalIMh2MKBV3dM84q5n9Kx+f15A4s5xUznL/nRqU6X+9BQOT10EJWkB7STR3gaI8TLboo8QwtqLGWw=="
  content_md5       = "d1222496e070b4d11793edfc0d2964fb"
  content_sha1      = "69a952ec47068e0cdd74140bfda354f8c2c08e49"
  content_sha256    = "37954def75697d6ea73f05b6d638219f04f7e15fa86a8782fe568ac3ce5cb3473"
  content_sha512    = "593bc85062ec6a588c87630a06fddd33ce2ae67f4ac7e7f5e40e2ce715199a5fe746a53a5fef4140e4f5384256901ed24eb13781a23c4e56e8a3c430b6a94637"
  directory_permission = "0777"
  file_permission   = "0777"
  filename          = ".nisha-terraform-lab.txt"
  id                = "69a952ec47068e0cdd74140bfda354f8c2c08e49"
}
```

Outputs:

```
created_file_content = "This file was created by Terraform using variables."
created_file_path    = ".nisha-terraform-lab.txt"
mouli@localhost:~/iac-demo/terraform$ ls -la
total 40
drwxr-xr-x 3 mouli mouli 4096 Sep 16 11:12 .
drwxr-xr-x 5 mouli mouli 4096 Sep 16 11:07 ..
drwxr-xr-x 3 mouli mouli 4096 Sep 16 11:08 .terraform
-rw-r--r-- 1 mouli mouli 1257 Sep 16 11:08 .terraform.lock.hcl
-rw-r--r-- 1 mouli mouli 256 Sep 16 11:07 main.tf
-rwxr-xr-x 1 mouli mouli 51 Sep 16 11:12 nisha-terraform-lab.txt
-rw-r--r-- 1 mouli mouli 250 Sep 16 11:07 outputs.tf
-rw-r--r-- 1 mouli mouli 1914 Sep 16 11:12 terraform.tfstate
-rw-r--r-- 1 mouli mouli 110 Sep 16 11:07 terraform.tfvars
-rw-r--r-- 1 mouli mouli 305 Sep 16 11:07 variables.tf
```

Terraform destroy

```
mouli@localhost: /iac-demo/terraform$ terraform destroy
local_file.lab_file: Refreshing state... [id=69a952ec47068e0cdd74140bfda354f8c2c08e49]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  - destroy

Terraform will perform the following actions:

# local_file.lab_file will be destroyed
- resource "local_file" "lab_file" {
  - content      = "This file was created by Terraform using variables." -> null
  - content_base64sha256 = "NSV091YH1upz8FttY4L58E9+FfqGqHgv5WisPOXLNHH=" -> null
  - content_base64sha512 = "WtVtUIGLsalIh2MKBy3dM84q5n9Kx+F15A4s5xUZmL/nRqU6X+9BQOT10EJWkB7STR3gaI8TLboo8QwtqLGNw==" -> null
  - content_md5      = "d1222496e070b4d11793edfc0d2964fb" -> null
  - content_sha1      = "69a952ec47068e0cdd74140bfda354f8c2c08e49" -> null
  - content_sha256     = "37954ef75607d6ea73f05b6d6382f9f04f7e15fa86a8782fe568ac3ce5cb3473" -> null
  - content_sha512     = "593bc85062ec6a588c87630a06fddd33ce2ae67f4ac7e7f5e40e2ce715199a5fe746a53a5fef4140e4f5384256901ed24eb13781a23c4e56e8a3c430b6a94637" -> null
  - directory_permission = "0777" -> null
  - file_permission      = "0777" -> null
  - filename             = "/nisha-terraform-lab.txt" -> null
  - id                   = "69a952ec47068e0cdd74140bfda354f8c2c08e49" -> null
}

Plan: 0 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  - created_file_content = "This file was created by Terraform using variables." -> null
  - created_file_path    = "/nisha-terraform-lab.txt" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

local_file.lab_file: Destroying... [id=69a952ec47068e0cdd74140bfda354f8c2c08e49]
local_file.lab_file: Destruction complete after 0s

Destroy complete! Resources: 1 destroyed.
```

Terraform state resource

```
mouli@localhost: /iac-demo/terraform$ terraform state list
local_file.lab_file
mouli@localhost: /iac-demo/terraform$ terraform show
# local_file.lab_file:
resource "local_file" "lab_file" {
  content      = "This file was created by Terraform using variables."
  content_base64sha256 = "NSV091YH1upz8FttY4L58E9+FfqGqHgv5WisPOXLNHH="
  content_base64sha512 = "WtVtUIGLsalIh2MKBy3dM84q5n9Kx+F15A4s5xUZmL/nRqU6X+9BQOT10EJWkB7STR3gaI8TLboo8QwtqLGNw=="
  content_md5      = "d1222496e070b4d11793edfc0d2964fb"
  content_sha1      = "69a952ec47068e0cdd74140bfda354f8c2c08e49"
  content_sha256     = "37954ef75607d6ea73f05b6d6382f9f04f7e15fa86a8782fe568ac3ce5cb3473"
  content_sha512     = "593bc85062ec6a588c87630a06fddd33ce2ae67f4ac7e7f5e40e2ce715199a5fe746a53a5fef4140e4f5384256901ed24eb13781a23c4e56e8a3c430b6a94637"
  directory_permission = "0777"
  file_permission      = "0777"
  filename             = "/nisha-terraform-lab.txt"
  id                   = "69a952ec47068e0cdd74140bfda354f8c2c08e49"
}

Outputs:
created_file_content = "This file was created by Terraform using variables."
created_file_path    = "/nisha-terraform-lab.txt"
mouli@localhost: /iac-demo/terraform$ ls -la terraform.tfstate
-rw-r--r-- 1 mouli mouli 1914 Sep 16 13:01 terraform.tfstate
```

Git protection

```
mouli@localhost:~/iac-demo/terraform$ ls -la terraform.tfstate
-rw-r--r-- 1 mouli mouli 1914 Sep 16 13:01 terraform.tfstate
mouli@localhost:~/iac-demo/terraform$ cd ..
mouli@localhost:~/iac-demo$ nano .gitignore
mouli@localhost:~/iac-demo$ cat .gitignore
.terraform/
*.tfstate
*.tfstate.*
crash.log
mouli@localhost:~/iac-demo$ git check-ignore -v terraform.tfstate
.gitignore:2:*.tfstate terraform.tfstate
```

inventory.ini

```
[local]
localhost ansible_connection=local
```

Ansible Inventory

```
mouli@localhost:~/iac-demo/ansible$ ansible-inventory -i inventory.ini --list
{
  "_meta": {
    "hostvars": {
      "localhost": {
        "ansible_connection": "local"
      }
    },
    "profile": "inventory_legacy"
  },
  "all": {
    "children": [
      "ungrouped",
      "local"
    ]
  },
  "local": {
    "hosts": [
      "localhost"
    ]
  }
}
```

playbook.yml

```
- name: Configure local machine with Ansible
  hosts: local
  become: true
  gather_facts: true

  tasks:
    - name: Install Nginx package
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Create deployment user
      ansible.builtin.user:
        name: deployuser
        state: present
        shell: /bin/bash
        create_home: true

    - name: Ensure Nginx service is started and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

Ansible playbook

```
mouli@localhost: ~/iac-demo/ansible$ ansible-playbook -i inventory.ini playbook.yml --ask-become-pass
BECOME password:

PLAY [Configure local machine with Ansible] *****
*****

TASK [Gathering Facts] *****
*****
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.12', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Install Nginx package] *****
*****
ok: [localhost]

TASK [Create deployment user] *****
*****
ok: [localhost]

TASK [Ensure Nginx service is started and enabled] *****
*****
ok: [localhost]

PLAY RECAP *****
localhost : ok=4 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

Ansible Playbook Execution

```
mouli@localhost:~/iac-demo/ansible$ id deployuser
uid=1005(deployuser) gid=1008(deployuser) groups=1008(deployuser)
mouli@localhost:~/iac-demo/ansible$ systemctl status nginx --no-pager
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-16 11:33:00 UTC; 4min 37s ago
     Docs: man:nginx(8)
  Process: 18754 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 18756 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 18757 (nginx)
    Tasks: 9 (limit: 4488)
  Memory: 7.2M (peak: 8.2M)
     CPU: 34ms
  CGroup: /system.slice/nginx.service
          └─18757 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
            └─18759 "nginx: worker process"
              └─18760 "nginx: worker process"
                └─18761 "nginx: worker process"
                  └─18762 "nginx: worker process"
                    └─18763 "nginx: worker process"
                      └─18764 "nginx: worker process"
                        └─18765 "nginx: worker process"
                          └─18766 "nginx: worker process"

Sep 16 11:33:00 localhost systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Sep 16 11:33:00 localhost systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```